Project - Low Level Design

on

Finance Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tarang Purohit	EN22CS3011033
2	Suhani Maheshwari	EN22CS301988
3	Smita Kumawat	EN22CS301961
4	Sneha Agrawal	EN22CS301963
5	Somya Neema	EN22CS301973

Group Name: Group 03D9

Project Number: DO-03

Industry Mentor Name:

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2025-26

1. Introduction

The Finance Support Triage System is an AI-powered application designed to automate the classification and handling of financial support queries. The system uses Large Language Models (LLMs) to analyze incoming user messages, determine intent and urgency, extract key financial entities, and generate appropriate responses.

Unlike traditional support systems, this solution reduces manual workload by providing instant AI-driven triage and response generation. The system ensures faster resolution and improved efficiency in handling finance-related queries.

1.1 Scope of the Document

This Low Level Design (LLD) document provides a detailed technical specification of the Finance Support Triage Agent — a production-grade reactive agentic AI system designed to automate the initial response workflow for finance-related communications. The document covers internal architecture, class design, data models, process flows, information flows, entity relationships, and technology justification.

1.2 Intended Audience

Audience	Purpose
Development Team	Implementation reference for classes, APIs, and data flow
Academic Evaluators	LLD presentation covering all five required presentation criteria
Industry / University Mentor	Review of design decisions and technology justifications
QA Team	Understand component boundaries for test case design

1.3 Definitions and Acronyms

Term	Meaning
LLM	Large Language Model
RAG	Retrieval Augmented Generation
API	Application Programming Interface
NER	Named Entity Recognition
CrewAI	Multi-agent AI orchestration framework

1.4 System Overview

The **Finance Support Triage System** is an AI-powered application that automates the analysis of financial support queries. It uses a multi-agent architecture powered by Large Language Models (LLMs) to classify user intent, detect urgency, extract key entities, and generate appropriate responses.

The system follows a full-stack architecture with a **React-based frontend**, a **FastAPI backend**, and a **MongoDB database**. User queries entered through the dashboard are sent to the backend, where a CrewAI-based multi-agent pipeline processes the request.

Each agent performs a specific task such as classification, extraction, decision-making, and response generation. The final structured output is stored in MongoDB and displayed on the frontend.

To ensure reliability, the system includes a fallback mechanism that generates mock responses when the external API is unavailable.

Overall, the system improves efficiency by enabling real-time, automated triage of finance-related queries.

2. System Design

2.1 Application Design

The system is decomposed into seven primary classes following the Single Responsibility Principle. Each class has a well-defined role within the pipeline.

2.1.1 Class Inventory

Class Name	Module	Responsibility
TriageRequest	models.py	Pydantic model — validates and structures the incoming finance query payload
TriageResult	models.py	Pydantic model — holds urgency level, intent, extracted entities, and draft response
ExtractedEntities	models.py	Pydantic model — structured container for all NER output fields
ClassificationEngine	classifier.py	Classifies query urgency (Critical/High/Medium/Low) and intent category via Gemini
EntityExtractor	ner_engine.py	Executes NER via Gemini to extract IDs, dates, amounts, and account numbers from text
ResponseGenerator	response_gen.py	Calls Gemini 2.5 Flash to produce a contextual draft response from classification and entities
TriageOrchestrator	agent.py	Central reactive agent — coordinates all three engines and assembles the final TriageResult

2.1.2 Class Description

TriageRequest (models.py)

The entry-point data contract for the entire system. It is a Pydantic BaseModel that validates every incoming finance message before it enters the agent pipeline. It holds the unique message identifier, the raw text body, the source channel (email, chat, or ticket), the timestamp, and an optional metadata dictionary for additional context.

TriageResult (models.py)

The output contract produced at the end of every agent run. It is a Pydantic BaseModel that captures the full triage outcome: the assigned urgency level (Critical, High,

Medium, or Low), the detected intent category, the nested ExtractedEntities object, the LLM-generated draft response, a confidence score, and a processing timestamp.

ExtractedEntities (models.py)

A nested Pydantic model embedded inside TriageResult. It holds all Named Entity Recognition (NER) output as typed lists: transaction IDs, account numbers, dates, monetary amounts, and customer identifiers. Having a dedicated model ensures all NER fields are validated and structured before use.

ClassificationEngine (classifier.py)

Responsible solely for determining the urgency and intent of an incoming message. It accepts the raw text, constructs a structured classification prompt, and calls the Gemini 2.5 Flash API.

The JSON response is parsed and returned as a ClassificationOutput object containing the urgency level, intent category, and a confidence score.

EntityExtractor (ner_engine.py)

Performs Named Entity Recognition on the raw message text. It builds a precise NER prompt instructing Gemini to return a JSON object containing all identifiable finance entities. The response is parsed and returned as an ExtractedEntities Pydantic model, ready to be embedded in the final TriageResult.

ResponseGenerator (response_gen.py)

Generates the contextual draft reply to send to the customer. It combines the original message, the classification output, and the extracted entities into a rich context prompt, then calls Gemini 2.5 Flash. The returned text is a professional, intent-aware draft response tailored to the specific finance query.

TriageOrchestrator (agent.py)

The central reactive agent and the single entry-point for all triage processing. It coordinates the three engines in sequence: first it classifies the message, then it extracts entities, and finally it generates the draft response. All three outputs are assembled into a TriageResult and returned to the UI layer.

FastAPIController (main.py)

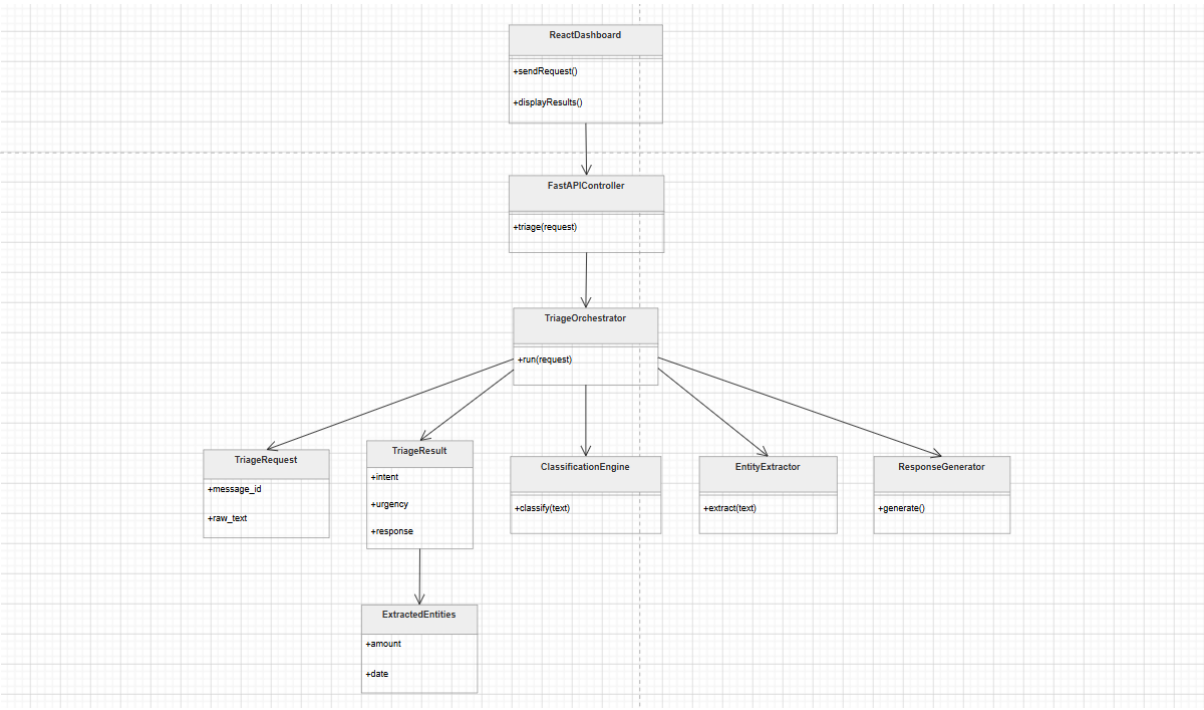
Acts as the backend API layer of the system. It receives HTTP requests from the React frontend, validates incoming data using Pydantic models, and invokes the TriageOrchestrator for processing. The structured TriageResult is then returned as a JSON response to the client and can be stored in MongoDB for persistence.

React Dashboard (frontend)

The frontend layer of the application built using React and Tailwind CSS. It provides an interactive user interface where users can input finance queries, view triage results, and manage tickets. It communicates with the backend via REST APIs and displays structured outputs such as urgency, intent, extracted entities, and generated responses.

2.1.3 Class Diagram

Class Diagram — Finance Support Triage Agent



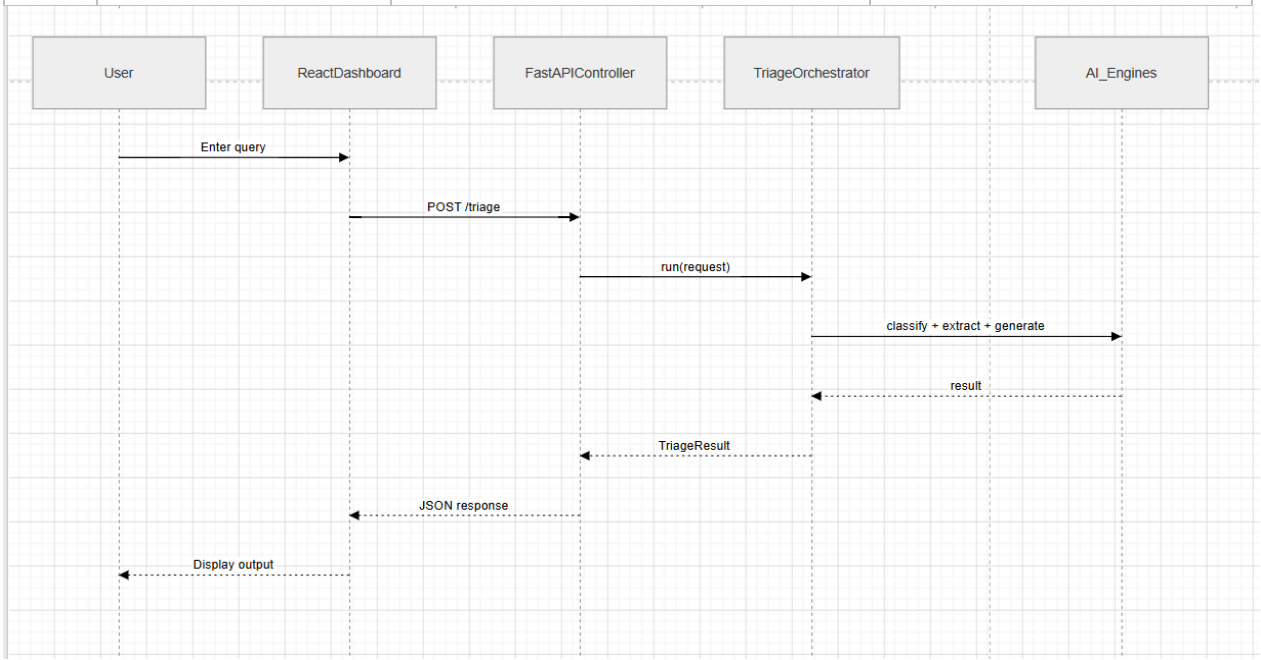
2.2 Process Flow — User Flow

This section describes the complete end-to-end flow from a human agent submitting a finance message to receiving a structured triage result.

2.2.1 User Flow Steps

Step	Actor	Action	Output
1	Human Agent	Opens UI in browser; enters finance support message text	Text submitted to backend

Step	Actor	Action	Output
2	GradioUI	Validates input; creates TriageRequest Pydantic object with metadata	TriageRequest object
3	TriageOrchestrator	Receives TriageRequest; calls ClassificationEngine.classify()	Forwards raw_text to engine
4	ClassificationEngine	Constructs classification prompt; sends to Gemini 2.5 Flash API; parses response	ClassificationOutput (urgency + intent)
5	TriageOrchestrator	Calls EntityExtractor.extract()	Forwards raw_text to NER engine
6	EntityExtractor	Constructs NER prompt; sends to Gemini 2.5 Flash; parses JSON response	ExtractedEntities object
7	TriageOrchestrator	Calls ResponseGenerator.generate() with text + classification + entities	Forwards context to generator
8	ResponseGenerator	Builds context-rich prompt; calls Gemini 2.5 Flash; returns draft response	Draft reply string
9	TriageOrchestrator	Assembles TriageResult Pydantic model from all three outputs	TriageResult object
10	Human Agent	Reviews draft; edits if necessary; approves and dispatches to customer	Final response sent to customer



2.3 Information Flow

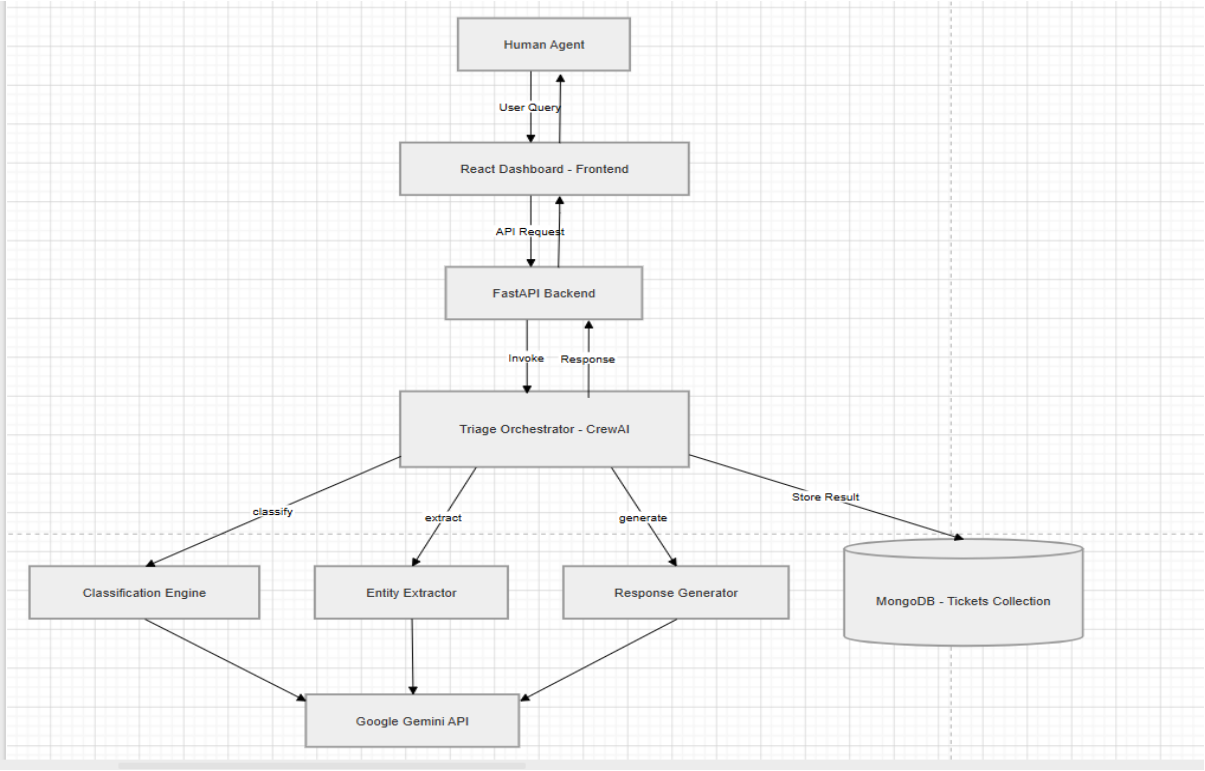
This section details how data transforms as it passes through each component of the system.

2.3.1 Data Transformation Table

Stage	Input	Process	Output	Format
1. Ingestion	Raw finance text (string)	Pydantic validation	TriageRequest object	Pydantic model
2. Classification	TriageRequest.raw_text	LLM prompt → Gemini API	ClassificationOutput	JSON → Pydantic
3. NER Extraction	TriageRequest.raw_text	NER prompt → Gemini API	ExtractedEntities	JSON → Pydantic
4. Response Gen.	text + classification + entities	Context prompt → Gemini API	Draft response string	Plain text
5. Assembly	All three outputs above	TriageResult constructor	TriageResult object	Pydantic model
6. UI Rendering	TriageResult object	Gradio component binding	Human-readable panels	Gradio UI widgets
7. Analytics	Historical TriageResult records	Pandas DataFrame aggregation	Trend report / CSV export	pandas DataFrame

2.3.2 Information Flow Diagram

Information Flow Diagram — Finance Support Triage Agent



2.4 Components Design

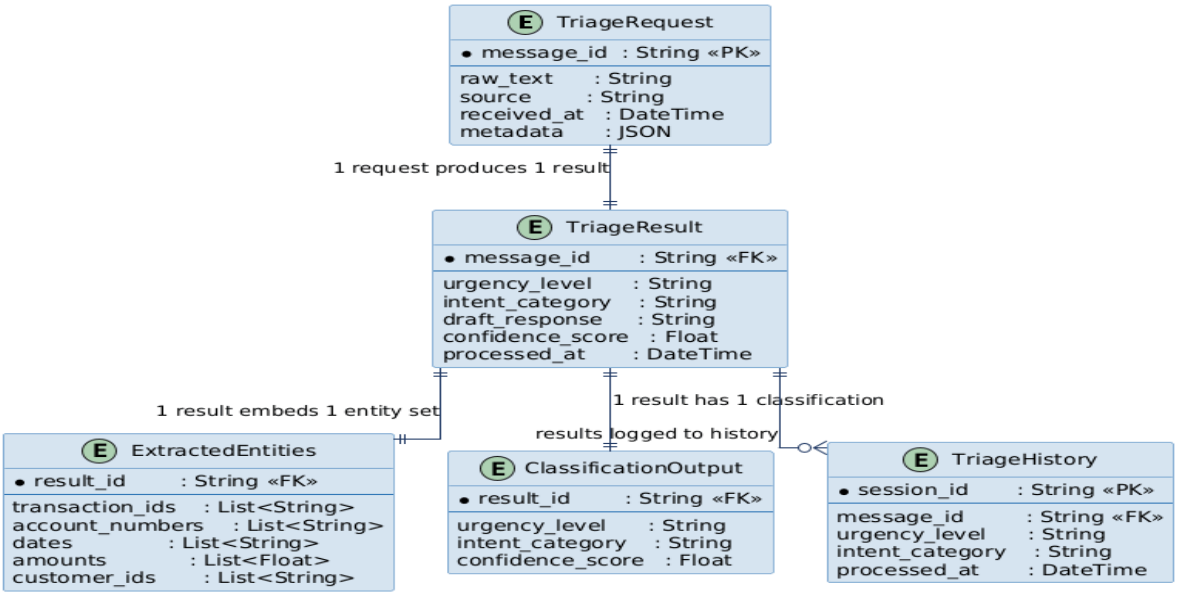
2.4.1 Entity List

Entity	Type	Key Attributes	Role in System
TriageRequest	Pydantic Model	message_id, raw_text, source, received_at, metadata	Input contract; validates all incoming data
TriageResult	Pydantic Model	message_id, urgency_level, intent_category, entities, draft_response, processed_at	Output contract; complete triage output from the agent
ExtractedEntities	Pydantic Model	transaction_ids, account_numbers, dates, amounts, customer_ids	NER output; embedded in TriageResult
ClassificationOutput	Pydantic Model	urgency_level, intent_category, confidence_score	Intermediate result from ClassificationEngine
GeminiClient	Service Client	api_key, model_name	Abstraction layer over Gemini 2.5 Flash HTTP calls

Entity	Type	Key Attributes	Role in System
TriageOrchestrator	Agent Controller	classifier, extractor, generator	Orchestrates all engines; single entry-point for UI
TriageHistory	Pandas DataFrame	session_id, message_id, urgency_level, intent_category, processed_at	In-memory analytics store for trend reporting

2.4.2 Entity Relationship Diagram

Entity Relationship Diagram — Finance Support Triage Agent



2.5 Key Design Considerations

2.5.1 Agentic AI Design

The system implements a Reactive Agent architecture. The three-engine design mirrors the Sense–Reason–Act cycle of agentic AI systems:

Agent Phase	Engine	LLM Call Description
SENSE	ClassificationEngine	Perceives message intent and urgency from raw text
REASON	EntityExtractor	Extracts structured facts (entities) from the message
ACT	ResponseGenerator	Generates a contextually accurate action (draft reply)

2.5.2 Error Handling Strategy

- Pydantic validation at ingestion catches malformed inputs before they reach the LLM.
- All Gemini API calls are wrapped in try/except; fallback urgency defaults to 'High' on failure.
- JSON parsing of LLM responses uses safe parsing with default empty models on failure.
- Gradio UI surfaces user-friendly error messages for backend failures.

2.5.3 Scalability Considerations

- Stateless design: each TriageRequest is processed independently with no session state.
- GeminiClient is an abstraction layer and can be swapped for any LLM provider (Open/Closed Principle).
- Pandas history store can be replaced with a SQL or NoSQL database for production-scale persistence.
- Gradio async queuing supports multiple concurrent submissions without blocking.

2.6 Technology Stack & Justification

Technology	Version	Role in System	Justification
Python	3.x	Primary language for all backend logic and orchestration	Industry-standard for AI/ML; rich ecosystem for LLM integration, data processing, and web frameworks
Google Gemini 2.5 Flash	API	LLM for classification, NER, and response generation	State-of-the-art multimodal LLM; Flash variant provides low latency ideal for real-time triage; strong structured JSON output capability
Pydantic	v2.x	Data validation and schema enforcement across all models	Enforces strict input/output contracts; eliminates runtime type errors; seamless JSON serialisation and deserialisation between components
Pandas	2.x	Historical triage data processing and analytics reporting	Powerful DataFrame operations for aggregating triage history, computing urgency distributions, and generating trend reports; CSV export support

2.7 API Catalogue

2.7.1 Internal Python APIs

Method Signature	Class	Input Type	Output Type	Description
run(request)	TriageOrchestrator	TriageRequest	TriageResult	Main agent entry point; coordinates all engines
classify(text)	ClassificationEngine	str	ClassificationOutput	Returns urgency level and intent category
extract(text)	EntityExtractor	str	ExtractedEntities	Returns all NER entities extracted from text
generate(text,cls,ent)	ResponseGenerator	str, output, entities	str	Returns contextual draft response string
complete(prompt)	GeminiClient	str	str	Abstraction over raw Gemini API HTTP call
triage(request)	FastAPIController	JSON	JSON	Handles POST /triage request and returns result
get_tickets()	FastAPIController	—	List	Fetches stored tickets from MongoDB

2.7.2 External API

API	Endpoint	Method	Auth	Called By	Usage
Google Gemini 2.5 Flash	/v1beta/models/gemini-2.5-flash:generateContent	POST	API Key	All 3 Engines	Classification, NER, and response generation inference

3. References

Sr No	Reference	URL / Source
1	Google Gemini API Documentation	https://ai.google.dev/api/generate-content
2	Pydantic v2 Documentation	https://docs.pydantic.dev/latest/
3	Gradio Documentation	https://www.gradio.app/docs
4	Pandas Documentation	https://pandas.pydata.org/docs/
5	PlantUML Language Reference	https://plantuml.com/guide
6	Reactive Agent Architecture	Russell & Norvig, Artificial Intelligence: A Modern Approach, Ch. 2
7	Medicaps University Datagami Course	Internal course material and LLD template